CardKeep Package Overview

Version 0.6.2 · October 2026 · github.com/AxialForge/cardkeep

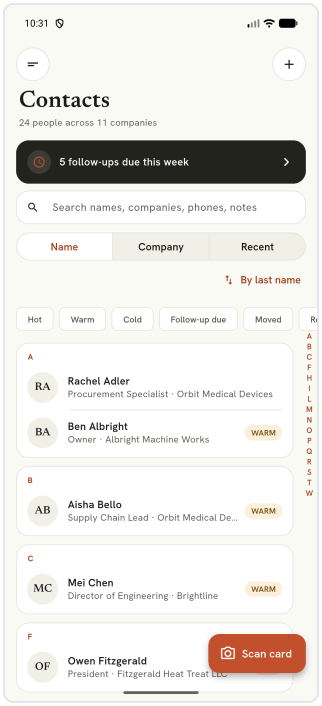
This overview explains CardKeep three times, each in more depth: a short version for anyone, a medium version for people deciding whether to use it, and an advanced version for developers and reviewers.

Part 1: The simple version

What it is. CardKeep is a phone app that turns paper business cards into contacts.

What it's for. You collect cards at meetings and trade shows. They pile up in a drawer and the people on them are hard to find later. With CardKeep you take a photo of each card; the app reads the name, job, company, phone numbers and email, and saves them as a contact with the card attached. Later you can search for anyone, call them, set a reminder to follow up, and see who you met at which event.

How it works. Everything happens on your phone. The app reads the card itself instead of sending the picture to a website, and your contacts are never uploaded anywhere. There is no account to create. The only time it uses the internet is to check for a newer version, and only when you ask.



Part 2: The medium version

What it does

Area	What you get
Scanning	Live card outline, hints, automatic capture; one card, a batch, or a pile in one photo; gallery photos; back sides; QR codes
Reading	On-device text recognition, fields filled in, unsure fields marked; tap-to-fix and read-a-region
Contacts	Search across every field, filters, A–Z index, lead ratings, notes, tags, the card itself
Follow-ups	Due dates, reminders on the day, drafted follow-up emails
Events	Trade shows as scan sessions, goals, "met at", follow up with everyone
Duplicates and careers	Possible duplicates with reasons and a merge screen; job changes and retirements tracked
NFC and My card	Read NFC business cards; share your own details as a QR code or contact file
Data in and out	vCard and CSV import/export (Google, Outlook, HubSpot, Salesforce, LinkedIn), full backup zip and restore

Who it's for

People who meet many others in person: sales, purchasing, engineers at suppliers and customers, anyone working trade shows. It suits someone who wants their network on their own phone rather than in a company CRM or a cloud service.

Privacy in practice

- No account, no analytics, no ads. Google's ML Kit telemetry is switched off.
- Data lives in the app's private storage, excluded from Android cloud backup and device transfer.
- Deleted contacts are wiped after 7 days. Photos are saved without location data.
- The app goes online only for update checks, and verifies downloads against GitHub's checksums.

Limits today

Android only. Installed from GitHub, not Google Play. No app lock, no sync with a computer, and the on-phone database is not encrypted (it relies on Android's app sandbox). These are on the roadmap.

Part 3: The advanced version

Stack

Layer	Choice
UI	Flutter 3.47 (Dart 3), Material with custom themes, go_router, Riverpod 3
Storage	SQLite via drift 2.35 (schema v3, migrations from v1), files in app-private storage
Text recognition	Google ML Kit Text Recognition (bundled Latin model, on device)
Barcodes	ML Kit Barcode Scanning (QR: vCard, MECARD, URLs)
Image processing	OpenCV (opencv_dart / dartcv4, built from source) for card detection and perspective correction; `image` package for encoding
Camera	`camera` plugin (CameraX on Android)
NFC	nfc_manager 4 + NDEF records
Reminders	flutter_local_notifications 22.3 with inexact alarms and boot receivers
Phone numbers	phone_numbers_parser (region-aware validation and formatting)

Code map

Folder	Holds
`lib/app`	App shell, router, theme, drawer, providers, start-up jobs (restore result, recovery, purge, backup)
`lib/scan`	Scan pipeline and background worker
`lib/parser`	Turns OCR lines into fields (rules, confidence, casing, QR merge)
`lib/core/db`	drift schema, migrations
`lib/core/*`	Repositories: contacts, cards queue, career, duplicates, events, follow-ups, tags, images, transfer (vCard/CSV/backup), update, links, settings
`lib/features/*`	Screens per area
`lib/nfc`	NFC reading

Scan pipeline

1. **Capture** (camera or gallery). The JPEG is written to `cards/pending/` *before* any processing, so a kill never loses a capture; the camera's own temp file is deleted.
2. **Detect** the card quadrilateral with OpenCV (edges, contours, corner ordering). Live hints come from the same detector at low resolution: size, sharpness (Laplacian variance), steadiness across frames, glare and exposure.
3. **Rectify**: perspective-correct to at least 1400 px on the long side; dark cards get a contrast-boosted second pass.
4. **Read**: ML Kit text recognition plus barcode scan on the rectified image.
5. **Parse**: rule-based field extraction (names, titles, companies, phones with extensions and fax, emails, URLs, addresses, extras), input capped at 200 characters per line and 80 lines per side to keep regexes linear. Each field gets a confidence from the OCR confidence, the rule that matched and cross-checks; a contact QR overrides text.
6. **Queue**: the card row is committed to the review queue; only then is the pending file removed. Clear cards in Batch and Pile can be saved straight to contacts. A capture that fails three times moves to `cards/pending/failed/`.

Images are content-addressed (SHA-256) under `cards/original`, `rectified`, `thumb` and `logo`, with EXIF stripped.

Data model and lifecycle

People, contact points, roles (person ↔ company with dates and status), companies, cards, tags, events, follow-ups, change suggestions, successions and a full-text search table. Times are stored as Unix seconds. Delete is a tombstone (`deleted_at`) for undo and future sync; after 7 days a purge blanks the person row, deletes their details, cards and search entry, and removes image files nothing references.

Backup format

One zip: `card_organizer.sqlite` (a consistent `VACUUM INTO` copy with deleted people purged), the referenced card images, and `manifest.json` (app, version, schema, created, counts, SHA-256 per file). Restore checks the manifest, every hash, size limits (1 GB zip, 2 GB unpacked), rejects unlisted files and path escapes, and runs `PRAGMA integrity_check` on the staged database. The swap happens at the next start, before the database opens, as a journalled sequence (`old` → `new` → `keep` → `done`) that resumes after a kill and never deletes the only copy of anything; the previous data is kept in `before_restore/` for 30 days.

Network and trust

The only network code is the updater: it reads the latest GitHub release, requires the asset's SHA-256 digest, caps the download at 200 MB, verifies the hash and hands the APK to Android's package installer. ML Kit's transport backend registration is removed from the merged manifest, so its telemetry has no destination. `allowBackup=false` with data-extraction rules excluding everything. Incoming vCard intents are accepted only from `content://` URIs outside the app's own providers. Clipboard copies of contact details are marked sensitive; `debugPrint` is silent in release.

Build and release

R8 shrinking with keep rules for ML Kit. Release signing requires `key.properties` (no debug-key fallback); v2 + v3 signatures. CI (GitHub Actions, actions pinned to commit SHAs) runs analyze and tests on every push; a `vX.Y.Z` tag builds split APKs and publishes them with `SHA256SUMS.txt`, the R8 mapping and Dart debug symbols.

Quality

180 automated tests (unit, widget and data-layer), including a simulated kill at every step of a restore, schema upgrade v1 → v3, hostile-input timing tests for the CSV and OCR parsers, and a ground-truth set of real card OCR. A structured mobile release audit (OWASP MASVS-mapped) verified 34 fixes for 0.6.2: the GitHub channel is a Conditional Go; Google Play is pending (Play build without the updater, AAB, privacy policy, Data safety form).